Brian Wonjun Lee

Philadelphia, PA | 215-294-7152 | leebwj@sas.upenn.edu | leebrian.dev | LinkedIn

Education

University of Pennsylvania — Philadelphia, PA
MSE in Computer Graphics & Game Technology (CGGT), Submatriculation — *Expected May 2028*
BA in Computer Science & Design (Double Major), Benjamin Franklin Scholars — *Expected May 2027*
Relevant Coursework: Interactive Computer Graphics, Computer Animation, 3-D Computer Modeling, Introduction to Algorithms, Introduction to Computer Systems, Programming Languages & Techniques

Experience

Software Engineer Intern, Aleph Lab (Y Combinator) — *June 2026 – Aug 2026 · San Francisco (Remote)*

- **Stopped a release that would have shipped to zero users** — the planned over-the-air update pointed at a runtime a dependency upgrade had invalidated: it would have deployed, reported success and reached nobody. Caught it against the live binaries' fingerprints and rerouted to a store build
- **Unblocked a population that could never have been tested** — the rollout flag was scoped by email, which cannot resolve before sign-in, so signed-out and first-time users could never receive the new experience, including the app's own login and first-class screens
- **Gave the company a deploy-free way to undo its entire visual layer** — reverting a UI regression used to mean a store release and days of review; it is now one switch at any percentage, over a tested byte-identical off-path
- **Stopped the app telling parents their child had done nothing when the network failed** — two dashboard screens drew a failed request identically to an empty one, so a parent's read on their own kid came from a dropped request
- **Turned a channel that went dark whenever nobody approved a send into one that runs itself** — proposing, clearing its own guardrails and reaching **91% of targeted devices** unattended, producing the win-back's first ever revenue at **~2.7× the reach** once I proved its eligibility logic had excluded most of its audience
- **Made a children's app safe from notification injection** — anyone with a user ID could push arbitrary content until I bearer-authed four public endpoints, **zero caller changes**

Software Engineer Intern, BitMango (Puzzle1 Studio) — *June 2025 – Aug 2025 · Pangyo, South Korea*

- **Kept 50+ live Unity mobile titles shipping** across a summer of releases, holding UI/UX and gameplay stable
- **Cleared 100+ QA-reported bugs into production**, in Unity prefabs and C# across a portfolio no single engineer owned
- **Kept the portfolio compliant and in the stores** through a platform requirement change, upgrading SDKs fleet-wide

Project Lead & Product Designer, Penn Spark — *Jan 2025 – Present · Philadelphia, PA*

- **Shipped client-facing 0-to-1 products** leading a cross-functional team of **15+** designers and developers

Database Engineer Intern, IT-Farm — *June 2022 – Aug 2022 · Seongnam, South Korea*

- **Made semiconductor data from 20+ client companies queryable** — including Samsung, SK and LG — by designing a SQL database prototype managing **1M+ entities** in Oracle and SQLite
- **Cut retrieval time across large-scale datasets** by restructuring and optimizing the queries the team relied on

Projects

Mini Minecraft | *C++, OpenGL, GLSL* — *Apr 2026*

- **Made a from-scratch voxel world read as varied rather than noisy** — **7-biome procedural generation** (Perlin/FBM/Voronoi) and **3D Perlin caves** in a C++/OpenGL engine (team of 3)
- **Made that world legible** — the GLSL pipeline I owned: PCF shadows, screen-space reflections, vertex AO, day-night cycle

Passenger | *Unreal Engine 5, HLSL, Maya* — *Mar 2026 – Apr 2026*

- **Gave a short film its entire visual identity** — 2 **HLSL post-process shaders** (anisotropic Kuwahara, cell shader) over GBuffer channels, delivered as a ray-traced 2560×1080 render
- **Brought its character to life** — rigged and animated in Maya (IK limbs, spline-IK spine) driving an Unreal Animation Blueprint

Mini Maya | *C++, OpenGL, Qt* — *Feb 2026 – Mar 2026*

- **Made arbitrary meshes editable in place** — a **half-edge structure** giving topology-aware traversal, plus an OBJ parser accepting arbitrary n-gons, rendered via OpenGL VBOs
- **Turned coarse geometry into smooth surfaces** with **Catmull–Clark subdivision** and topology ops (split, extrusion, triangulation) in a Qt editor

Technical Skills

Game Engines: Unity, Unreal Engine 5, C#, C++, Animation Blueprints, prefab & gameplay systems
Graphics & 3D: OpenGL, GLSL, HLSL, Maya, Blender, Substance Painter
Languages: C++, C#, Python, TypeScript, JavaScript, Java, SQL
Frameworks & Tools: React Three Fiber, Three.js, React, Node.js, Git, GitLab, VS Code